KPC Shipment Sync

Integration guide for your fulfillment dashboard

Version 1.1 · 20 October 2026 · Source: KPC Express website (kpcexpress.com)

What this does: when a KPC admin tags a shipment as **PH Warehouse** or **Delivered** on the KPC website, the shipment's details are **pushed to your system instantly**. You don't need to poll or ask for updates; they come to you.

< 1 second
from the admin's click to your server
(measured: 0.03 s)

2 events
PH Warehouse
Delivered

0 missed
automatic retries for about 7 hours,
plus a catch-up API

How an update reaches your dashboard

1. Admin tags
PH Warehouse or Delivered on the KPC website

›

2. KPC sends
an HTTPS POST to your URL, signed with your secret

›

3. You save
the shipment in your database and reply `200`

›

4. You refresh
your open dashboard screens (optional, your WebSocket/SSE)

KPC side (done) Your side (to build)

Real-time push, not polling

This is a *webhook*: KPC calls your server the moment the status changes. The Pull API (page 6) is only a **safety net** for your first data load and for anything missed while your server was down.

Contents

Quick start 2
Step-by-step integration 3
Complete example endpoint 4
What you receive 5
Pull API (first load and catch-up) 6
Testing and go-live 7
Troubleshooting and FAQ 8
Code in PHP and Python 9

Quick start

You send us

- **Webhook URL**, HTTPS, accepts `POST` + JSON
  e.g. https://dashboard.yourco.com/webhooks/kpc
- **Staging URL** (optional) for testing
- **Fields** you need (page 5)
- **History**: how many months of past data

We send you (secure channel, never by chat)

- **Signing secret**: proves a request came from KPC
- **API key**: for the Pull API
- **Pull API URL**
- Test events to your staging URL

What your endpoint must do

#	Do this	Why	
1	**Check the signature**	So nobody else can send you fake "Delivered" updates.	REQUIRED
2	**Ignore duplicates** by `event_id`	The same event can occasionally arrive twice.	REQUIRED
3	**Keep only newer data**	Compare `occurred_at`, so an older, retried event never overwrites newer data.	REQUIRED
4	**Reply `200` within 10 seconds**	Anything else is treated as failed and sent again.	REQUIRED
5	**Push to open screens**	So dashboards already open update without a refresh.	RECOMMENDED

One update, start to finish

```
# What KPC sends to your URL
POST /webhooks/kpc
Content-Type: application/json
X-KPC-Event:      shipment.delivered
X-KPC-Event-Id:   a824173b-a92d-430a-8868-c093e5b4be54
X-KPC-Timestamp:  1760942575
X-KPC-Signature:  sha256=5f0c2e…

{ "event": "shipment.delivered", "event_id": "a824173b-…", "occurred_at": "2026-10-20T06:42:55+00:00",
  "shipment": { "shipment_id": 14648, "shipment_code": "KPC 509509EC-9123", "delivery_status":
"Delivered",
               "ph_warehouse_tagged_at": "2026-10-19T02:10:03+00:00", "delivered_at": "2026-10-
20T06:42:55+00:00", … } }

# What you reply
HTTP/1.1 200 OK
```

Step-by-step integration

1 Create two tables in your database

One for shipments (what the dashboard shows), one for events (to skip duplicates).

```
CREATE TABLE kpc_shipments (
  shipment_id            BIGINT PRIMARY KEY,     -- KPC's ID, never changes
  shipment_code          TEXT,
  delivery_status        TEXT,
  ph_warehouse_tagged_at TIMESTAMPTZ,
  delivered_at           TIMESTAMPTZ,
  data                   JSONB,                  -- the full shipment object, for any other field
  last_event_at          TIMESTAMPTZ NOT NULL    -- occurred_at of the last event applied
);
CREATE TABLE kpc_events (
  event_id    UUID PRIMARY KEY,
  received_at TIMESTAMPTZ DEFAULT now()
);
```

PostgreSQL shown. MySQL works the same way: use `JSON`, `DATETIME` and `INSERT … ON DUPLICATE KEY UPDATE`.

2 Add a POST endpoint and verify the signature

Read the **raw body** (before JSON parsing). Compute `HMAC-SHA256(secret, timestamp + "." + rawBody)` in hex and compare it with `X-KPC-Signature` (after `sha256=`). Reject it if it doesn't match, or if the timestamp is more than 5 minutes old.

3 Skip duplicates

Insert the `event_id` into `kpc_events`. If it's already there, reply `200` and stop.

4 Save the shipment, newer data only

Upsert by `shipment_id`, but only when the event's `occurred_at` is newer than `last_event_at`.

```
INSERT INTO kpc_shipments (shipment_id, shipment_code, delivery_status, ph_warehouse_tagged_at,
                           delivered_at, data, last_event_at)
VALUES ($1, $2, $3, $4, $5, $6, $7)
ON CONFLICT (shipment_id) DO UPDATE SET
  shipment_code = EXCLUDED.shipment_code,   delivery_status = EXCLUDED.delivery_status,
  ph_warehouse_tagged_at = EXCLUDED.ph_warehouse_tagged_at,
  delivered_at = EXCLUDED.delivered_at,     data = EXCLUDED.data,
  last_event_at = EXCLUDED.last_event_at
WHERE kpc_shipments.last_event_at < EXCLUDED.last_event_at;   -- older events can't overwrite
```

5 Reply `200` and refresh open dashboards

Reply right away. Then, if your dashboard uses WebSockets, SSE, Pusher, or Firebase/Supabase Realtime, broadcast the updated shipment so open screens update live. Any slow work (reports, totals) runs after the reply.

6 Load existing data once, then add a safety net

At go-live, load what's already in PH Warehouse through the Pull API snapshot (page 6). Then call the event feed every few minutes or once a day, from the last `seq` you saved, to recover anything missed during downtime.

Complete example endpoint (Node.js)

Copy this, set the two environment variables, and adapt the database calls to your stack. It covers steps 2–5. PHP and Python versions are on page 9.

```
import express from "express";
import crypto from "node:crypto";
import pg from "pg";

const SECRET = process.env.KPC_WEBHOOK_SECRET;   // from KPC, server-side only
const db = new pg.Pool({ connectionString: process.env.DATABASE_URL });
const app = express();

function isFromKpc(raw, ts, sig) {
  const expected = "sha256=" + crypto.createHmac("sha256",
SECRET).update(`${ts}.${raw}`).digest("hex");
  const fresh = Math.abs(Date.now() / 1000 - Number(ts)) < 300;           // within 5 minutes
  return fresh && sig.length === expected.length &&
         crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(expected));
}

// express.raw = keep the exact bytes KPC sent (needed for the signature)
app.post("/webhooks/kpc", express.raw({ type: "application/json" }), async (req, res) => {
  const raw = req.body.toString("utf8");
  if (!isFromKpc(raw, req.get("X-KPC-Timestamp") ?? "", req.get("X-KPC-Signature") ?? "")) {
    return res.status(401).send("invalid signature");                      // step 2
  }

  const { event_id, occurred_at, shipment: s } = JSON.parse(raw);

  const dup = await db.query(
    "INSERT INTO kpc_events (event_id) VALUES ($1) ON CONFLICT DO NOTHING", [event_id]);
  if (dup.rowCount === 0) return res.sendStatus(200);                     // step 3: already processed

  await db.query(`
    INSERT INTO kpc_shipments (shipment_id, shipment_code, delivery_status,
                               ph_warehouse_tagged_at, delivered_at, data, last_event_at)
    VALUES ($1, $2, $3, $4, $5, $6, $7)
    ON CONFLICT (shipment_id) DO UPDATE SET
      shipment_code = EXCLUDED.shipment_code, delivery_status = EXCLUDED.delivery_status,
      ph_warehouse_tagged_at = EXCLUDED.ph_warehouse_tagged_at, delivered_at = EXCLUDED.delivered_at,
      data = EXCLUDED.data, last_event_at = EXCLUDED.last_event_at
    WHERE kpc_shipments.last_event_at < EXCLUDED.last_event_at`,              // step 4
    [s.shipment_id, s.shipment_code, s.delivery_status,
     s.ph_warehouse_tagged_at, s.delivered_at, s, occurred_at]);

  res.sendStatus(200);                                                    // step 5: reply fast
  liveUpdate(s);                        // optional: refresh open dashboards
});

app.listen(3000);
```

The most common mistake

Using `express.json()` (or any body parser) before the webhook route. It changes the body, and the signature will never match. Use `express.raw()` on this route as shown.

What you receive

Two events: `shipment.ph_warehouse` (status changed **to** PH Warehouse) and `shipment.delivered` (status changed **to** Delivered).

Headers

Header	Meaning
`X-KPC-Event`	Event type, same as `event` in the body.
`X-KPC-Event-Id`	Unique ID. A retry of the same event keeps the same ID.
`X-KPC-Timestamp`	Unix seconds when this attempt was sent.
`X-KPC-Signature`	`sha256=` + hex HMAC-SHA256 of `timestamp + "." + raw body`.

> **Good to know**
>
> - Other statuses aren't sent. A shipment that's reverted and tagged again sends a new event. Bulk tagging sends one event per shipment.
> - Every body has the **full shipment**, not only what changed.
> - Times are ISO 8601 in **UTC**; add 8 hours for Manila time.
> - Any field can be `null`. **Ignore fields you don't know**; we may add more.
> - `delivered_at` exists from go-live onward. Older shipments may have it empty.

Shipment fields

Field	Type	Meaning
`shipment_id`	integer	**Primary key.** Never changes.
`shipment_code`	text	Code shown to customers, e.g. `KPC 509509EC-9123`
`delivery_status`	text	`PH Warehouse` or `Delivered` at the time of the event
`ph_warehouse_tagged_at`	time · UTC	When first tagged PH Warehouse
`delivered_at`	time · UTC	When tagged Delivered
`payment_status`	text	`Pending`, `For Review`, `Paid`, …
`client_code` · `client_name` · `business_name`	text	Customer
`tracking_number` · `waybill_number` · `container_van`	text	Reference numbers
`quantity` · `box_quantity`	number	Item and box counts
`total_cbm` · `total_kgs`	number	Volume and weight
`source_warehouse` · `pickup_warehouse`	text	Origin and pickup warehouse names
`item_description`	text[]	Items in the shipment
`created_at`	time · UTC	When the shipment was created
`date_received`	time · no zone	Date received, as entered by the admin

Pull API (first load and catch-up)

Read-only. Every call needs the header `x-api-key: <your API key>`; without it you get `401`. `{BASE_URL}` is provided at go-live.

Use it for	Call
First load: what's in PH Warehouse right now	`GET {BASE_URL}/snapshot?status=PH%20Warehouse`
Catch-up: every event since your last one	`GET {BASE_URL}?after_seq=<last seq>`

Snapshot: current shipments in a status

```
curl "{BASE_URL}/snapshot?status=PH%20Warehouse&after_id=0&limit=500" -H "x-api-key: $KPC_API_KEY"

{ "status": "PH Warehouse",
  "shipments": [ { "shipment_id": 14648, "shipment_code": "KPC 509509EC-9123", … } ],
  "next_after_id": 14648,
  "has_more": false }
```

`status` can be `PH Warehouse`, `Ready For Pickup`, `Warehouse Overstaying` or `Delivered`. While `has_more` is `true`, call again with `after_id=next_after_id`. `limit` is 1–1000, default 500.

Event feed: everything since a point

```
curl "{BASE_URL}?after_seq=0&limit=500" -H "x-api-key: $KPC_API_KEY"

{ "events": [
    { "seq": 1, "event_id": "dc4a079a-…", "event": "shipment.ph_warehouse", "occurred_at": "…",
"shipment": { … } },
    { "seq": 2, "event_id": "a824173b-…", "event": "shipment.delivered",   "occurred_at": "…",
"shipment": { … } } ],
  "next_after_seq": 2,
  "has_more": false }
```

Each event has the same body and `event_id` as the webhook, so feed each one through the **same handler**. Save `next_after_seq`, and use it in the next call.

> **Simple catch-up loop**
> `seq = saved_seq or 0` → call the feed with `after_seq=seq` → process each event like a webhook → save `next_after_seq` → repeat while `has_more`. Run it every few minutes or once a day.

Testing and go-live

1. Check your signature code with this test vector

Before we connect anything, run your verification code with these exact values. It must produce the expected signature.

Input	Value
Secret	`kpc_test_secret_do_not_use`
Timestamp	`1760918400`
Raw body	`{"event": "shipment.ph_warehouse", "event_id": "11111111-2222-3333-4444-555555555555", "shipment": {"shipment_id": 1001, "shipment_code": "KPC TEST-0001"}, "occurred_at": "2026-10-20T00:00:00+00:00"}`
Expected	`sha256=b52505688e7186e3b94465d8d9be73aab8944da4d6561777c4d95882fa2639cf`

```
# check it from a terminal
printf '%s' '1760918400.{"event": "shipment.ph_warehouse", "event_id": "11111111-2222-3333-4444-555555555555", "shipment": {"shipment_id": 1001, "shipment_code": "KPC TEST-0001"}, "occurred_at": "2026-10-20T00:00:00+00:00"}' \
  | openssl dgst -sha256 -hmac 'kpc_test_secret_do_not_use'
```

The timestamp in this test is old, so turn off the 5-minute check while you test it.

2. Go-live checklist

✓	Step	Who
☐	Send staging + production webhook URLs, the field list and the history range	You
☐	Send the signing secret, API key and Pull API URL securely	KPC
☐	Build the endpoint; pass the test vector above	You
☐	We tag test shipments on KPC staging → you confirm they appear on your staging dashboard	Both
☐	We switch production events to your production URL	KPC
☐	First load with `/snapshot`, then start the catch-up feed from `after_seq=0`	You
☐	Historical data (if needed): agreed separately, delivered as a one-time export	Both

3. Delivery and retries

Your server replies	What KPC does
`2xx` within 10 s	Done. The event is marked delivered.
Anything else, a timeout, or offline	Retries after about 2, 4, 8, 16 and 32 min, then hourly: **12 tries, about 7 hours**. After that, the event is still in the Pull API.

Troubleshooting and FAQ

Problem	Fix
Signature never matches	Sign the **raw** body, not parsed and re-serialized JSON. Use `timestamp + "." + body`. Check the secret has no extra spaces or line breaks. Test with the vector on page 7.
Some updates are missing	Your server was down or replied non-2xx. Run the catch-up feed from your last `seq`; every event is there.
Same event received twice	Normal now and then. Skip it by `event_id`.
A delivered shipment shows PH Warehouse	An older event overwrote a newer one. Only apply an event when `occurred_at` is newer.
Dashboard doesn't change until refresh	Your data is up to date; your screen isn't. After saving, push the update to open pages (step 5, page 3).
Pull API returns 401	The header must be exactly `x-api-key`. Ask us if the key may have been rotated.
Status changed, but no event	Only changes **to** PH Warehouse or Delivered are sent. Use `/snapshot` for other statuses.

FAQ

Is it real-time? Yes. KPC pushes to your server the moment the admin confirms. In testing it took 0.03 seconds.

Do we need to poll? No. The Pull API is only for the first load and for catching up after downtime.

Why not a direct database connection or a Supabase key? That would expose all KPC data, including other customers' personal information. With the webhook you receive only the fields agreed for your dashboard.

Why a webhook and not a WebSocket? A WebSocket loses any update made while you're disconnected. A webhook is retried until your server accepts it, so nothing is missed. You can still use WebSockets *inside* your system to update open screens.

How do we compute fulfillment rate? Both timestamps come with every shipment. For example: *delivered this month ÷ arrived at PH Warehouse this month*, or the *average days from* `ph_warehouse_tagged_at` *to* `delivered_at`. Use your team's definition.

Can we get more fields? Yes. Tell us which ones and we'll add them. Your code should already ignore unknown fields.

What if a secret or key leaks? Tell us right away. We'll issue a new one, and the old one stops working.

> **Need help?**
> Contact the KPC development team with the `event_id` or `shipment_code`, the time it happened, and your server's response. We can resend specific events, add fields, change your URL, or rotate keys.

Code in PHP and Python

PHP (Laravel or plain PHP)

```php
$raw = file_get_contents('php://input');                      // raw body, before json_decode
$ts  = $_SERVER['HTTP_X_KPC_TIMESTAMP'] ?? '';
$sig = $_SERVER['HTTP_X_KPC_SIGNATURE'] ?? '';

$expected = 'sha256=' . hash_hmac('sha256', $ts . '.' . $raw, getenv('KPC_WEBHOOK_SECRET'));
if (abs(time() - (int)$ts) > 300 || !hash_equals($expected, $sig)) {
    http_response_code(401); exit('invalid signature');
}

$event = json_decode($raw, true);
$s     = $event['shipment'];

// 1. skip duplicates
$stmt = $pdo->prepare('INSERT INTO kpc_events (event_id) VALUES (?) ON CONFLICT DO NOTHING');
$stmt->execute([$event['event_id']]);
if ($stmt->rowCount() === 0) { http_response_code(200); exit; }

// 2. upsert, newer only (same SQL as page 3)
$pdo->prepare($UPSERT_SQL)->execute([$s['shipment_id'], $s['shipment_code'], $s['delivery_status'],
    $s['ph_warehouse_tagged_at'], $s['delivered_at'], json_encode($s), $event['occurred_at']]);

http_response_code(200);
```

In Laravel, use `$request->getContent()` for the raw body and exclude the route from CSRF protection.

Python (FastAPI)

```python
import hmac, hashlib, os, time, json
from fastapi import FastAPI, Request, Response

app = FastAPI()
SECRET = os.environ["KPC_WEBHOOK_SECRET"].encode()

def is_from_kpc(raw: bytes, ts: str, sig: str) -> bool:
    expected = "sha256=" + hmac.new(SECRET, f"{ts}.".encode() + raw, hashlib.sha256).hexdigest()
    return abs(time.time() - int(ts or 0)) < 300 and hmac.compare_digest(expected, sig)

@app.post("/webhooks/kpc")
async def kpc_webhook(request: Request):
    raw = await request.body()                                # raw bytes
    if not is_from_kpc(raw, request.headers.get("x-kpc-timestamp", ""),
                       request.headers.get("x-kpc-signature", "")):
        return Response(status_code=401)
    event = json.loads(raw)
    if already_processed(event["event_id"]):                  # INSERT … ON CONFLICT DO NOTHING
        return Response(status_code=200)
    upsert_newer_only(event["shipment"], event["occurred_at"])   # same SQL as page 3
    return Response(status_code=200)
```